

Árbol de fundamentos antes de OAuth

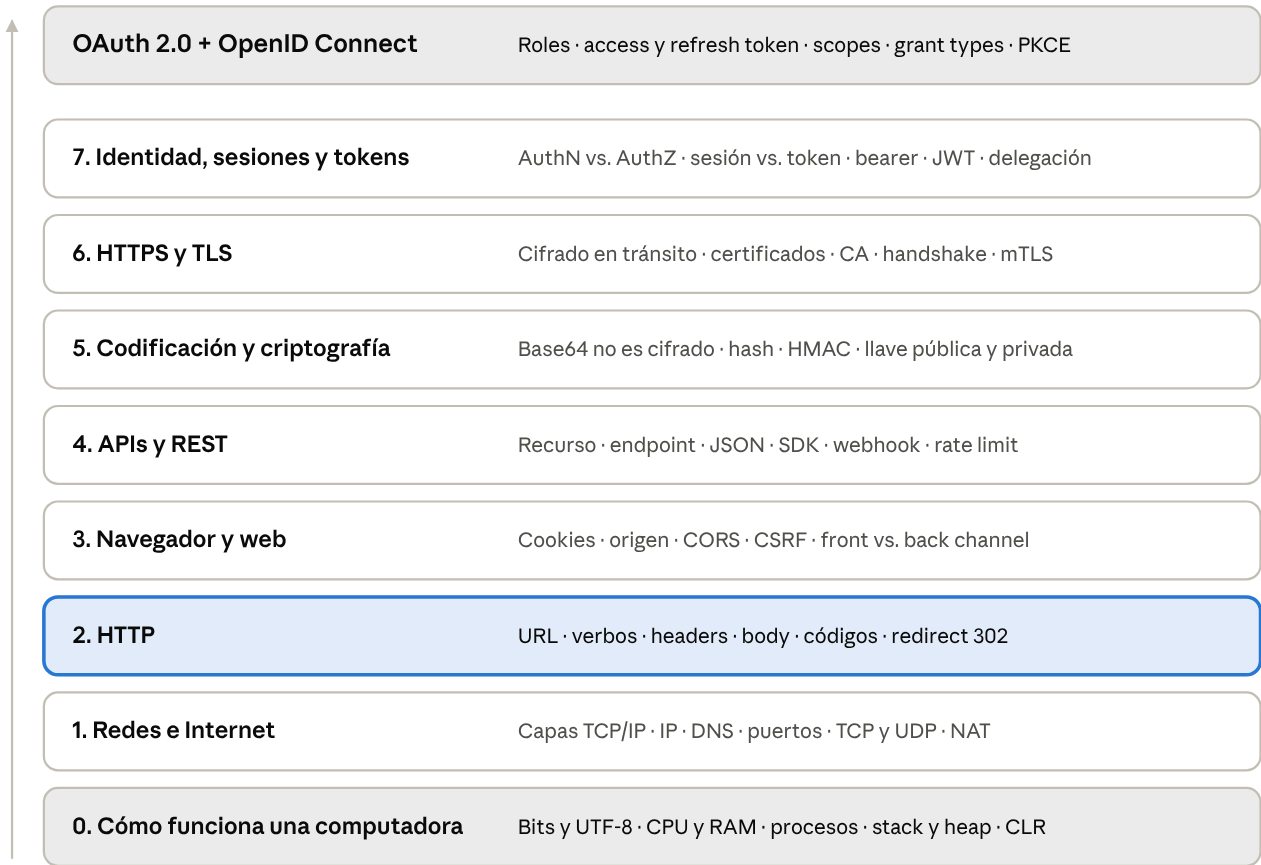
Sep 30, 2026 · @Alex

OAuth es la punta de un árbol de 8 niveles. La raíz es cómo funciona una computadora (nivel 0); encima vienen redes, HTTP, navegador, APIs, criptografía, HTTPS e identidad. Muchos ya los usas a diario en DiSí; lo que falta es ponerles nombre y entender por qué funcionan. El árbol completo son unas 28 horas de teoría, repartidas en las semanas 1 a 8 del plan reestructurado.

El árbol completo

OAuth descansa sobre ocho niveles; HTTP es el que más pesa

Se estudia de abajo hacia arriba: cada nivel usa las palabras del anterior



Árbol de prerrequisitos · 8 niveles hasta OAuth

Cada franja depende de las de abajo. El nivel 2 va resaltado porque OAuth está hecho casi por completo de peticiones HTTP y redirecciones. Aparte de este árbol, necesitas C# para programar el ejercicio, que ya está en tu plan de 12 semanas.

Nivel 0: cómo funciona una computadora

Es la raíz de todo el árbol, y también de C#. Aquí se explica por qué existen el stack y el heap, qué es un proceso que "escucha" en un puerto o por qué un texto se ve con símbolos raros. Tiene cuatro ramas.

0.1 Datos: todo son bits

Concepto	Lo que debes poder explicar	Dónde lo verás
Bit y byte	Un bit vale 0 o 1; un byte son 8 bits y guarda 256 valores posibles (0 a 255)	Tamaño de archivos, <code>byte[]</code> en C#
Sistema binario	Contar en base 2: $1011 = 8 + 0 + 2 + 1 = 11$	Permisos, flags, máscaras de red
Hexadecimal	Base 16 (0-9, A-F); un byte se escribe con 2 dígitos: <code>FF</code> = 255	Hashes, colores <code>#FF0000</code> , GUIDs
Tipos numéricos	Un <code>int</code> ocupa 4 bytes, un <code>long</code> 8. Por eso tienen límites y pueden desbordarse	<code>int</code> , <code>long</code> , <code>decimal</code> en C#
Punto flotante	<code>double</code> guarda aproximaciones: $0.1 + 0.2$ no da exactamente 0.3 . Para dinero se usa <code>decimal</code>	Cálculos de crédito e intereses
Codificación de caracteres	Cada letra es un número. ASCII cubre inglés (1 byte); Unicode cubre todos los idiomas y UTF-8 es la forma estándar de guardarlo	Acentos rotos (Ã³) en CSV o respuestas de API

0.2 Hardware: dónde viven los datos

Concepto	Lo que debes poder explicar	Dónde lo verás
CPU	Ejecuta instrucciones, una tras otra, a gran velocidad. Tiene varios núcleos que trabajan en paralelo	Por qué existe el paralelismo
RAM (memoria)	Rápida y volátil: se borra al apagar. Ahí viven los programas mientras corren	Variables y objetos de C#
Almacenamiento (disco)	Lento pero permanente: archivos y bases de datos	SQL Server guarda en disco y cachea en RAM
Jerarquía de velocidad	Registros > caché > RAM > SSD > red. Cada salto es mucho más lento	Por qué una consulta a BD o a una API cuesta tanto
Dirección de memoria	Cada byte de RAM tiene una dirección; una referencia es, en esencia, una dirección	Tipos por referencia en C#

0.3 Sistema operativo: quién administra todo

Concepto	Lo que debes poder explicar	Dónde lo verás
Sistema operativo	Reparte CPU, memoria, disco y red entre los programas	Windows Server de DiSí
Proceso	Un programa en ejecución, con su propia memoria aislada	<code>w3wp.exe</code> (el proceso de IIS)
Hilo (thread)	Una línea de ejecución dentro de un proceso; un proceso puede tener muchos	<code>async/await</code> , <code>Task</code> , bloqueos
Stack y heap	Stack: memoria rápida y ordenada para llamadas a métodos y variables locales. Heap: memoria para objetos que viven más tiempo	Tipos por valor vs. por referencia

Concepto	Lo que debes poder explicar	Dónde lo verás
Sistema de archivos	Carpetas, rutas absolutas y relativas, permisos	Rutas en <code>web.config</code> , <code>appsettings.json</code>
Terminal / shell	Controlar el sistema con comandos de texto (PowerShell, bash)	<code>dotnet new</code> , <code>git</code> , <code>curl</code>
Variables de entorno	Valores de configuración que el SO entrega a los procesos	<code>ASPNETCORE_ENVIRONMENT</code> , secretos
Sockets y puertos	Un proceso pide al SO "escuchar" en un puerto; el SO le entrega lo que llegue ahí	Kestrel escuchando en el puerto 5000

0.4 Programas: del código a la ejecución

Concepto	Lo que debes poder explicar	Dónde lo verás
Compilado vs. interpretado	Compilado: se traduce antes de correr (C#, Go). Interpretado: se traduce al vuelo (JavaScript, Python)	Por qué C# marca errores al compilar
Cómo corre .NET	C# → compilador → IL (código intermedio en un <code>.dll</code>) → el CLR lo convierte a instrucciones de CPU con el JIT	Tus <code>.dll</code> en la carpeta <code>bin</code>
Garbage collector	El CLR libera solo la memoria de objetos que ya nadie usa	Por qué en C# no se libera memoria a mano
Librerías y dependencias	Código reutilizable empaquetado; en .NET son paquetes NuGet	<code>packages.config</code> , <code>.csproj</code>
Estructuras de datos	Arreglo, lista, diccionario (hash table), pila, cola, árbol. Cada una es rápida para algo distinto	<code>List<T></code> , <code>Dictionary<K,V></code> , <code>Queue<T></code>

Concepto	Lo que debes poder explicar	Dónde lo verás
Complejidad (Big O)	Cuánto crece el tiempo con los datos: $O(1)$ buscar en un diccionario, $O(n)$ recorrer una lista, $O(n^2)$ un ciclo dentro de otro	Por qué un ciclo anidado sobre 10 mil clientes se vuelve lento
Lógica booleana	AND, OR, NOT, tablas de verdad, cortocircuito (<code>&&</code>)	Toda condición <code>if</code>

Por qué importa: casi todas las preguntas "de fundamentos" de C# en entrevista (valor vs. referencia, `decimal` vs. `double` , `async` , `Dictionary` vs. `List`) tienen la respuesta en este nivel.

Nivel 1: redes e Internet

Es la base de todo: cómo una máquina encuentra a otra y le habla. Sin esto, términos como `localhost` , "puerto" o "servidor" suenan a magia.

Concepto	Lo que debes poder explicar	Dónde ya lo viste
Cliente / servidor	El cliente pide, el servidor responde. Un mismo programa puede ser ambos	Tu navegador (cliente) y el IIS de DiSí (servidor)
Dirección IP	El "domicilio" numérico de una máquina en la red (IPv4 <code>192.168.1.10</code> , IPv6)	Servidores de producción, reglas de firewall
DNS	Traduce un nombre (<code>disioperaciones.com</code>) a una IP. Es la agenda de contactos de Internet	Al configurar dominios de landings
Puerto	El "número de departamento" dentro de la máquina: 80 (HTTP), 443 (HTTPS), 1433 (SQL Server)	IIS Express en <code>localhost:44300</code>
localhost / 127.0.0.1	Tu propia máquina hablando consigo misma. Nunca sale a Internet	Al depurar en Visual Studio

Concepto	Lo que debes poder explicar	Dónde ya lo viste
TCP	Protocolo que garantiza que los datos lleguen completos y en orden. HTTP viaja sobre TCP	Invisible, pero siempre presente
Modelo en capas	Cada capa se apoya en la de abajo: red (IP) → transporte (TCP) → seguridad (TLS) → aplicación (HTTP)	Explica por qué "HTTPS es HTTP sobre TLS"
Firewall / proxy	Un filtro que permite o bloquea conexiones por IP, dominio o puerto	Lo que hoy bloquea a <code>googleapis.com</code> desde el entorno en la nube de Claude

Por qué importa para OAuth: tu consola abrirá un pequeño servidor en `127.0.0.1: <puerto>` para recibir la redirección de Google. Si no entiendes puertos y localhost, ese paso no tiene sentido.

A fondo: cómo viaja un dato por la red

El modelo **TCP/IP** organiza la red en 4 capas; el modelo **OSI** es una versión teórica de 7 capas que se menciona mucho en entrevistas. Cada capa envía sus datos dentro de la capa de abajo, como una carta dentro de un sobre dentro de una caja (**encapsulamiento**).

Capa TCP/IP	Capas OSI	Qué resuelve	Conceptos clave
Aplicación	7 Aplicación, 6 Presentación, 5 Sesión	Qué dicen los programas	HTTP, DNS, SMTP (correo), TLS
Transporte	4 Transporte	Entre qué programas y con qué garantías	Puertos , TCP (confiable, en orden, con <i>three-way handshake</i>), UDP (rápido, sin garantías: video, DNS)
Internet (red)	3 Red	Entre qué máquinas, a través de varias redes	IP , paquetes, routers , ruteo
Acceso a red	2 Enlace, 1 Física	Cómo pasa la señal dentro de una red local	Cable, Wi-Fi, Ethernet , dirección MAC , switch

Concepto	Lo que debes poder explicar
Paquete	Los datos se parten en trozos pequeños; cada uno lleva IP de origen y destino y puede tomar caminos distintos
Dirección MAC	Identificador físico de la tarjeta de red; sirve solo dentro de la red local
IP pública vs. privada	Privadas (<code>192.168.x.x</code> , <code>10.x.x.x</code>) solo existen dentro de tu red; la pública es la que ve Internet
Subred y máscara (CIDR)	<code>192.168.1.0/24</code> significa "las 256 direcciones que empiezan con <code>192.168.1</code> "; define qué máquinas están en la misma red
Router y gateway	El router conecta redes distintas; el <i>default gateway</i> es la puerta de salida de tu red local
NAT	Tu router traduce muchas IPs privadas a una sola pública. Por eso Internet no puede llamar directo a tu laptop
DHCP	Asigna IPs automáticamente al conectarte a una red
Resolución DNS completa	Caché local → servidor DNS del proveedor → servidores raíz → servidor del dominio <code>.com</code> → servidor autoritativo. Tipos de registro: A (IP), CNAME (alias), MX (correo), TXT (verificaciones)
Latencia y ancho de banda	Latencia: cuánto tarda un dato en llegar. Ancho de banda: cuántos datos caben por segundo
Herramientas	<code>ping</code> (hay conexión), <code>tracert</code> (por dónde pasa), <code>nslookup</code> (qué IP tiene un dominio), <code>netstat</code> (qué puertos están abiertos)

Ejercicio: corre `nslookup disiooperaciones.com`, `tracert google.com` y `netstat -ano` en PowerShell, y explica en voz alta qué te dice cada resultado.

Pregunta clásica de entrevista: "¿Qué pasa cuando escribes una URL en el navegador y presionas Enter?" La respuesta recorre DNS, TCP, TLS, HTTP y el renderizado. Al terminar los niveles 1 a 3 debes poder contarla completa.

Nivel 2: HTTP

Es el nivel más importante del árbol. OAuth está hecho casi por completo de peticiones HTTP, redirecciones y headers.

Concepto	Lo que debes poder explicar	Dónde ya lo viste
Request / response	Toda comunicación es un pedido y una respuesta. Nada más	Cada acción de un controller MVC
Anatomía de una URL	<code>https://host:puerto/ruta?clave=valor#fragmento</code> : esquema, host, puerto, path, query string y fragmento	Filtros por URL en el portal del cliente
Métodos (verbos)	GET lee, POST crea o envía, PUT reemplaza, PATCH modifica parte, DELETE borra	<code>[HttpGet]</code> y <code>[HttpPost]</code> en tus controllers
Headers	Metadatos de la petición o respuesta: <code>Content-Type</code> , <code>Authorization</code> , <code>Accept</code> , <code>Location</code> , <code>Set-Cookie</code>	Llamadas a Twilio y WhatsApp
Body	El contenido: JSON, formulario (<code>application/x-www-form-urlencoded</code>), archivo	Los payloads de los webhooks
Códigos de estado	2xx éxito, 3xx redirección, 4xx error del cliente, 5xx error del servidor. Memoriza 200, 201, 204, 301, 302, 400, 401, 403, 404, 409, 429, 500	Respuestas de tus APIs
Redirección (302 + <code>Location</code>)	El servidor le dice al navegador "ve a esta otra URL". Es el corazón de OAuth	<code>RedirectToAction</code> en MVC
Stateless	HTTP no recuerda peticiones anteriores; cada una llega sola. Por eso existen cookies y tokens	La razón de ser de la sesión
Idempotencia	Repetir la petición da el mismo resultado (GET, PUT, DELETE sí; POST no)	Reintentos de webhooks duplicados

Ejercicio: abre las herramientas de desarrollador del navegador (F12 → pestaña Network) en el portal de DiSí y analiza una petición: método, URL, headers, body y código de estado. Repite con un login para ver el `Set-Cookie`.

Nivel 3: el navegador y la web

En OAuth el navegador es un intermediario: lleva al usuario a Google y trae el code de vuelta. Hay que entender qué puede y qué no puede hacer.

Concepto	Lo que debes poder explicar	Dónde ya lo viste
Cookie	Un dato pequeño que el servidor guarda en el navegador con <code>Set-Cookie</code> , y que el navegador reenvía en cada petición a ese dominio	La cookie de sesión del portal
Atributos de cookie	<code>HttpOnly</code> (JavaScript no la lee), <code>Secure</code> (solo HTTPS), <code>SameSite</code> (no viaja en peticiones de otros sitios), expiración	Configuración de autenticación en <code>web.config</code>
Origen (origin)	Combinación de esquema + host + puerto. <code>https://a.com</code> y <code>https://api.a.com</code> son orígenes distintos	Llamadas del front React al backend
Same-Origin Policy	Una página no puede leer respuestas de otro origen. Es la defensa base del navegador	Errores de CORS en consola
CORS	Mecanismo para que un servidor permita explícitamente a otros orígenes leer sus respuestas	Headers <code>Access-Control-Allow-Origin</code>
Front channel vs. back channel	Front: datos que pasan por el navegador (visibles, manipulables). Back: petición directa servidor a servidor (privada)	Clave para entender por qué el code viaja por el navegador y los tokens no
CSRF	Un sitio malicioso hace que tu navegador envíe una petición a otro sitio donde ya tienes sesión	<code>@Html.AntiForgeryToken()</code> en MVC

Concepto	Lo que debes poder explicar	Dónde ya lo viste
XSS	Inyectar JavaScript en una página para robar datos (por ejemplo, tokens en <code>localStorage</code>)	Por qué se codifica la salida en Razor
Formulario HTML	<code><form method="post"></code> envía datos como <code>x-www-form-urlencoded</code> ; es el formato del token endpoint de OAuth	Todos tus formularios de onboarding

Nivel 4: APIs y REST

Aquí está la respuesta a tu duda sobre los "recursos": el concepto viene de REST.

Concepto	Lo que debes poder explicar	Dónde ya lo viste
API	Un contrato para que un programa use funciones o datos de otro sin conocer su interior	Twilio, Kobra, Ekatená, Google Maps
Web API	Una API que se consume por HTTP	Tus integraciones y tus propios endpoints
REST	Estilo de diseño: todo es un recurso con URL propia, y los verbos HTTP dicen qué hacerle	<code>GET /clientes/15/lineas</code>
Recurso	Cualquier cosa con identidad que la API expone: un cliente, una línea de crédito, un evento de calendario	Tus entidades en SQL Server
Endpoint	Una URL + método concretos de la API	<code>POST /api/solicitudes</code>

Concepto	Lo que debes poder explicar	Dónde ya lo viste
JSON	Formato de texto para datos: objetos <code>{}</code> , arreglos <code>[]</code> , strings, números, booleanos, <code>null</code>	Respuestas de todas tus APIs
Serialización	Convertir un objeto C# a JSON y viceversa (<code>System.Text.Json</code> , <code>Newtonsoft</code>)	<code>JsonConvert.SerializeObject</code>
SDK / librería cliente	Paquete que envuelve las llamadas HTTP de una API en métodos de tu lenguaje	<code>Twilio</code> en NuGet, <code>Google.Apis.Calendar.v3</code>
Webhook	Una API al revés: el proveedor llama a tu endpoint cuando ocurre algo	WhatsApp y Twilio en onboarding y cobranza
Rate limit	Límite de peticiones por tiempo; al pasarlo recibes 429	Cuotas de Google Maps
Documentación de API / OpenAPI	La especificación de endpoints, parámetros y respuestas; Swagger es la interfaz que la muestra	Docs de Twilio y Meta

Ejercicio: con Postman o `curl`, llama a una API pública sin autenticación (por ejemplo, `https://api.github.com/users/<tu-usuario>`) y describe la respuesta: código, headers y estructura del JSON.

Nivel 5: codificación y criptografía básica

No necesitas matemáticas, pero sí distinguir cinco operaciones que se confunden todo el tiempo. Esta es la brecha que más se nota en entrevista.

Operación	Qué hace	¿Reversible?	Ejemplo
Codificación (encoding)	Cambia el formato para que los datos viajen bien. No protege nada	Sí, sin llave	Base64, Base64URL, URL encoding (%20), UTF-8
Hash	Convierte cualquier dato en una huella de tamaño fijo. El mismo dato siempre da la misma huella	No	SHA-256. PKCE usa el hash del <code>code_verifier</code>
HMAC	Hash mezclado con una llave secreta: prueba que el mensaje no se alteró y que lo envió quien tiene la llave	No	Firma de webhooks de Twilio y Meta
Cifrado simétrico	Oculto datos con una llave que cifra y descifra	Sí, con la llave	AES
Cifrado asimétrico	Un par de llaves: la pública cifra y la privada descifra	Sí, con la llave privada	RSA, curvas elípticas
Firma digital	Al revés: la llave privada firma y cualquiera verifica con la pública	Se verifica, no se revierte	Firma de los JWT de Google (RS256)

Otros conceptos de este nivel:

- **Hash de contraseñas:** las contraseñas se guardan con hashes lentos y con **salt** (bcrypt, PBKDF2, Argon2), nunca cifradas ni en texto plano.
- **Número aleatorio criptográfico:** `state` y `code_verifier` deben generarse con `RandomNumberGenerator`, no con `Random`.
- **Certificado digital:** un documento que dice "esta llave pública pertenece a google.com", firmado por una **Autoridad Certificadora (CA)**.
- **Llaves públicas publicadas (JWKS):** el Authorization Server publica sus llaves públicas para que cualquiera verifique sus tokens.

Frase clave para entrevista: "Base64 no es cifrado. Un JWT está firmado, no cifrado: cualquiera puede leerlo, pero nadie puede alterarlo sin invalidar la firma."

Ejercicio: en una consola C#, calcula el SHA-256 de un texto y su Base64URL, y comprueba que un cambio de una letra produce un hash totalmente distinto.

Nivel 6: HTTPS y TLS

OAuth 2.0 eliminó las firmas de OAuth 1.0 y delegó toda la seguridad del transporte a HTTPS. Sin HTTPS, un bearer token viajaría a la vista de cualquiera.

Concepto	Lo que debes poder explicar	Dónde ya lo viste
HTTPS	HTTP dentro de un túnel cifrado por TLS. Mismos métodos y headers, pero ilegibles para terceros	Todos los sitios de DiSí
TLS	El protocolo que crea ese túnel. Garantiza confidencialidad, integridad y autenticidad del servidor	El candado del navegador
Handshake	Al conectar, el servidor presenta su certificado, el cliente lo verifica y ambos acuerdan una llave simétrica para la sesión (asimétrico para acordar, simétrico para transmitir)	Invisible, pero ocurre en cada conexión
Cadena de confianza	Tu sistema operativo trae una lista de CAs confiables; si el certificado viene firmado por una de ellas, se acepta	Errores de "certificado no válido"
Certificado en IIS	Se instala en el servidor y se asocia (binding) al puerto 443 del sitio	Renovaciones de SSL en producción
Man-in-the-middle	Alguien intercepta la comunicación entre cliente y servidor. TLS lo impide si se valida el certificado	Por qué nunca se desactiva la validación del certificado

Concepto	Lo que debes poder explicar	Dónde ya lo viste
mTLS	TLS donde también el cliente presenta certificado	APIs bancarias

Qué protege TLS y qué no: cifra la URL completa, headers y body mientras viajan. No protege lo que queda guardado en los extremos: el historial del navegador, logs del servidor y variables en memoria. Por eso los tokens nunca van en la URL.

Nivel 7: identidad, sesiones y tokens

El último escalón antes de OAuth. Aquí se juntan todos los niveles anteriores.

Concepto	Lo que debes poder explicar	Dónde ya lo viste
Identidad	Quién es un usuario para el sistema: un identificador único (id, email)	Tabla de usuarios del portal
Credencial	Lo que prueba la identidad: algo que sabes (contraseña), algo que tienes (celular, llave) o algo que eres (huella)	Login del portal
Autenticación (AuthN)	Verificar la credencial: "¿eres quien dices ser?"	Tu pantalla de login
MFA / 2FA	Pedir dos factores distintos	Códigos por WhatsApp o SMS
Autorización (AuthZ)	Decidir qué puede hacer alguien ya autenticado	<code>[Authorize(Roles = "Admin")]</code>
RBAC / permisos	Autorización por roles (analista, operaciones, admin) o por permisos finos	Módulos internos por área
Mínimo privilegio	Dar solo el acceso necesario	Usuarios de SQL con permisos limitados
Sesión (stateful)	El servidor guarda quién eres y el navegador solo lleva un id en una cookie	Sesión de ASP.NET MVC

Concepto	Lo que debes poder explicar	Dónde ya lo viste
Token (stateless)	El cliente lleva una credencial autocontenida o verificable en cada petición; el servidor no guarda estado	Tokens de Meta para WhatsApp
Bearer token	Token que funciona para quien lo porte, sin prueba adicional	Header <code>Authorization: Bearer ...</code>
Expiración y revocación	Un token caduca solo (<code>exp</code>) o se invalida antes de tiempo	Cerrar sesión, cambiar contraseña
JWT	Token en formato <code>header.payload.signature</code> , firmado (nivel 5) y codificado en Base64URL (nivel 5)	Decodifícalo en jwt.io
Delegación	Dar a otro un permiso limitado para actuar en tu nombre	El problema que resuelve OAuth
Federación / SSO	Confiar en otro sistema para autenticar a tus usuarios	"Iniciar sesión con Google"

Cuando domines este nivel, la definición de OAuth se lee casi sola: *delegación* (nivel 7) de acceso a *recursos* (nivel 4) mediante *redirecciones* (nivel 2) en el *navegador* (nivel 3), con *tokens* (nivel 7) protegidos por *HTTPS* (nivel 6) y *hashes* (nivel 5).

Orden de estudio y cómo encaja en el plan

Estudia de abajo hacia arriba: cada nivel usa palabras del anterior. Son unas 28 horas en total, a razón de 5.5 horas de teoría por semana.

Nivel	Horas	Semana del plan
0.1 Datos y 0.2 Hardware	4	1
0.3 Sistema operativo y 0.4 Programas	4	2
1. Redes e Internet	3	3
2. HTTP	3	4
3. Navegador y 4. APIs	4	5

Nivel	Horas	Semana del plan
5. Criptografía y 6. HTTPS	4.5	6
7. Identidad y tokens	2	7
OAuth 2.0 (la clase)	3.5	8

Cómo encaja: la teoría va los sábados (4 h) y el lunes (1.5 h); de martes a jueves es código en C#. Cada semana, el código practica lo que la teoría acaba de explicar. El calendario completo está en el plan de 12 semanas.

Cómo saber que dominas un nivel: puedes explicar cada concepto de su tabla en voz alta, en menos de 30 segundos y con un ejemplo propio, sin mirar. Agrega cada término a la nota 08 Glosario de tu bóveda.

- ☐ Nivel 0: cómo funciona una computadora
- ☐ Nivel 1: redes e Internet
- ☐ Nivel 2: HTTP
- ☐ Nivel 3: navegador y web
- ☐ Nivel 4: APIs y REST
- ☐ Nivel 5: codificación y criptografía
- ☐ Nivel 6: HTTPS y TLS
- ☐ Nivel 7: identidad y tokens
- ☐ Releer la clase de OAuth y contestar sus preguntas

Recursos

- [MDN: Visión general de HTTP](#), niveles 2 y 3
- [MDN: Cookies HTTP](#) y [CORS](#)
- [MDN: Códigos de estado](#)
- [Microsoft Learn: Criptografía en .NET](#), nivel 5
- [Cloudflare Learning: qué es TLS](#) y [qué es DNS](#), niveles 1 y 6
- [jwt.io](#), nivel 7
- [OWASP Cheat Sheets](#), para CSRF, XSS y almacenamiento de contraseñas

Para el nivel 0 y las redes a fondo:

- [CS50x de Harvard](#) (gratis, con subtítulos en español): semanas 0 a 5 para datos, memoria y estructuras de datos, y la semana de HTTP y redes
- [Microsoft Learn: Qué es .NET y cómo funciona](#), para CLR, IL, JIT y garbage collector
- [Cloudflare Learning Center](#), para capas, IP, NAT y DNS
- Libro *Code*, de Charles Petzold: explica desde el bit hasta la computadora sin requerir matemáticas